

Project 5 Chad Noyes

Chad Noyes

May 2026

1 Introduction

An LQR controller was used to control a DC motor using an STM32 running Simulink. A state space model of the motor was characterized in project 4 with the following state space matrices.

$$A = \begin{bmatrix} 0 & 1.0000 \\ 0 & -25.7909 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 401.9964 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad D = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

These matrices correspond to states of position θ and velocity ω , with input voltage V .

1.1 Position LQR

The position LQR controller used an augmented state space system, with state 1 replaced with error, $(\theta - r)$, where r is reference position in radians, state 2 remains velocity in radians per second, and state 3 is the integral of error. The following matrices are the result.

$$Aa = \begin{bmatrix} 0 & 1.0000 & 0 \\ 0 & -25.7909 & 0 \\ 1 & 0 & 0 \end{bmatrix}, \quad Ba = \begin{bmatrix} 0 \\ 401.9964 \\ 0 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad D = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

This augmented plant was discretized for a rate of 1000 Hz, and the following Q and R matrices were used to create a K matrix in Matlab. The dlqry command was used to compute the gain matrix.

$$Q = \begin{bmatrix} 1e8 & 0 & 0 \\ 0 & 1e3 & 0 \\ 0 & 0 & 1e6 \end{bmatrix}, \quad R = [1e6]$$

The resulting K matrix can be seen below.

$$K = [9.6866 \quad 0.1673 \quad 0.9663]$$

1.2 Velocity LQR

The velocity controller used 2 state LQR with states error ($e = \omega - r$), where r is ref velocity in rad/s, and integral of error.

$$Aa = \begin{bmatrix} 1.0000 & 0 \\ -25.7909 & 0 \end{bmatrix}, \quad Ba = \begin{bmatrix} 401.9964 \\ 0 \end{bmatrix}, \quad Ca = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad Da = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

This plant was discretized, and a discrete LQR controller was then created with the following weights.

$$Q = \begin{bmatrix} 1e3 & 0 \\ 0 & 1e7 \end{bmatrix}, \quad R = [1e6]$$

The resulting K matrix can be seen below.

$$K = [0.0795 \quad 3.1117]$$

2 Plant Simulation

2.1 Position Control

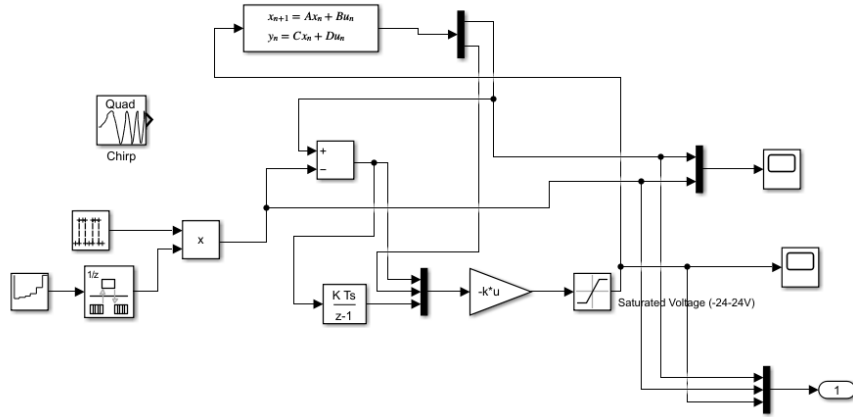


Figure 1: Simulink for Position Simulation

Figure 1 shows the Simulink model which was used to simulate the LQR controller for position control.

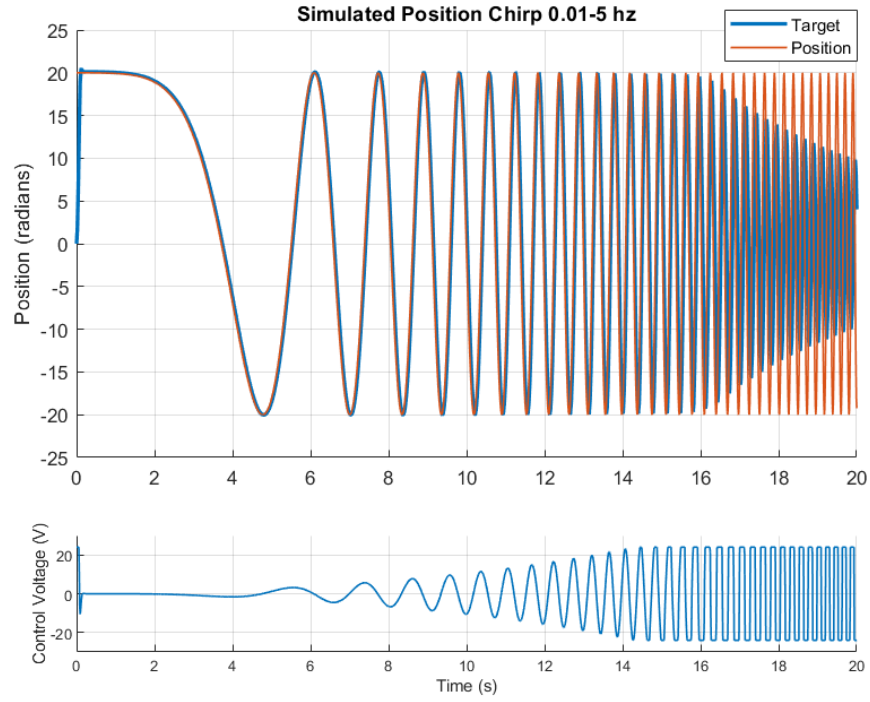


Figure 2: Chirp Simulation for Position Control

Figure 2 shows a chirp signal as a position control for the simulated motor. It can be seen that the controller closely follows the control for around 12 seconds which corresponds to a frequency around 4 Hz. The error increases as the frequency increases to 5 Hz.

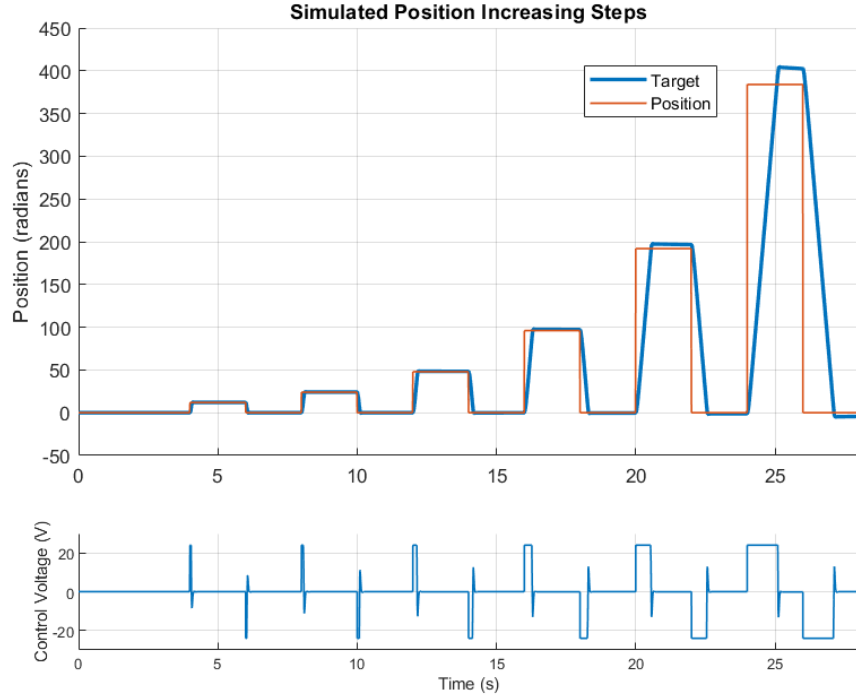


Figure 3: Increasing Step Simulation for Position Control

Figure 3 shows an increasing step simulation for position control. This simulation was used to tune the Q matrix, and was found to match very closely with the real simulations. The input voltage saturates at 24 V, but still is able to quickly reach the desired position at smaller steps with minimal steady state error. The experiment was designed to push the motor to its limit and large error is seen on the larger steps.

2.2 Velocity Control

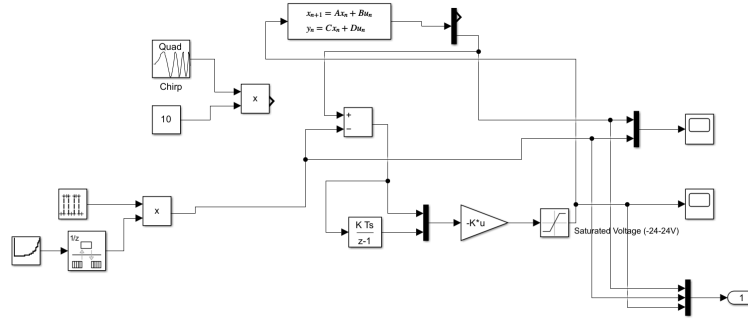


Figure 4: Simulink for Velocity Simulation

Figure 4 shows the Simulink model which was used to simulate the LQR controller for velocity control.

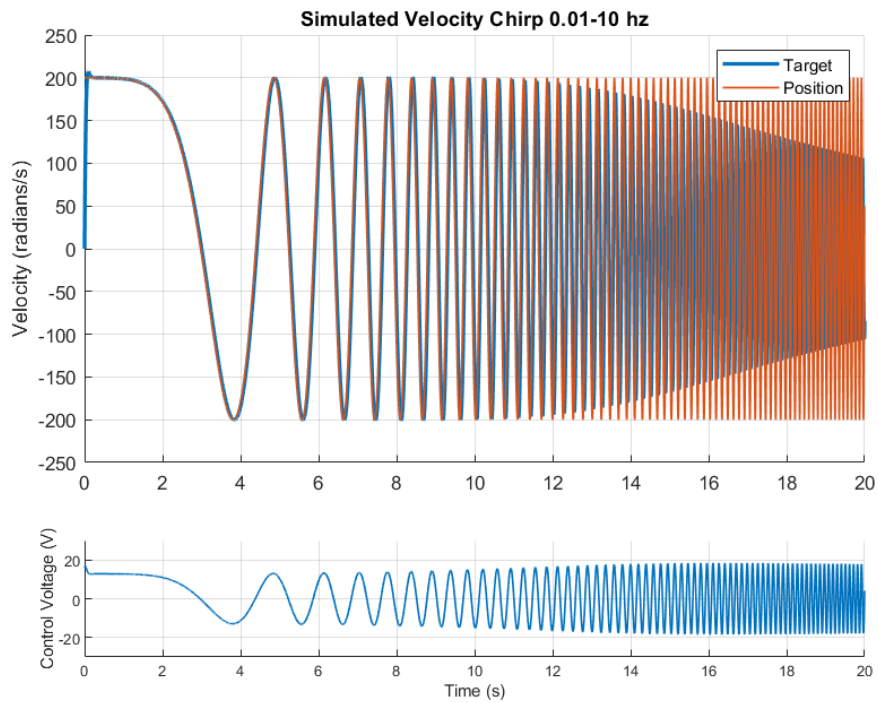


Figure 5: Chirp Simulation for Velocity Control

Figure 5 displays a 200 rad/s chirp from 0.01 to 10 Hz control signal for a simulated motor. Velocity can be changed quickly, so the motor is able to follow the control signal closely. There is small error at the higher frequencies.

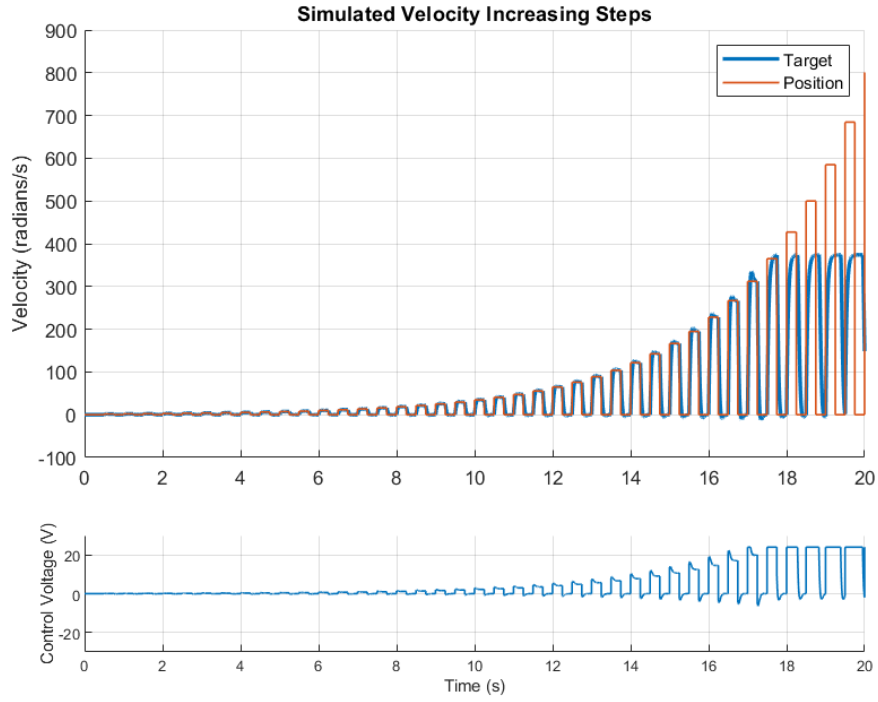


Figure 6: Increasing Step Simulation for Velocity Control

Figure 6 shows a simulation of the velocity step control for position. It can be seen that velocity can be quickly changed up to the maximum speed of the motor. At the maximum speed, the steady state voltage holds at 24 volts and no more power can be given. Otherwise there is quick motion to steps and low steady state error.

3 Power Connections

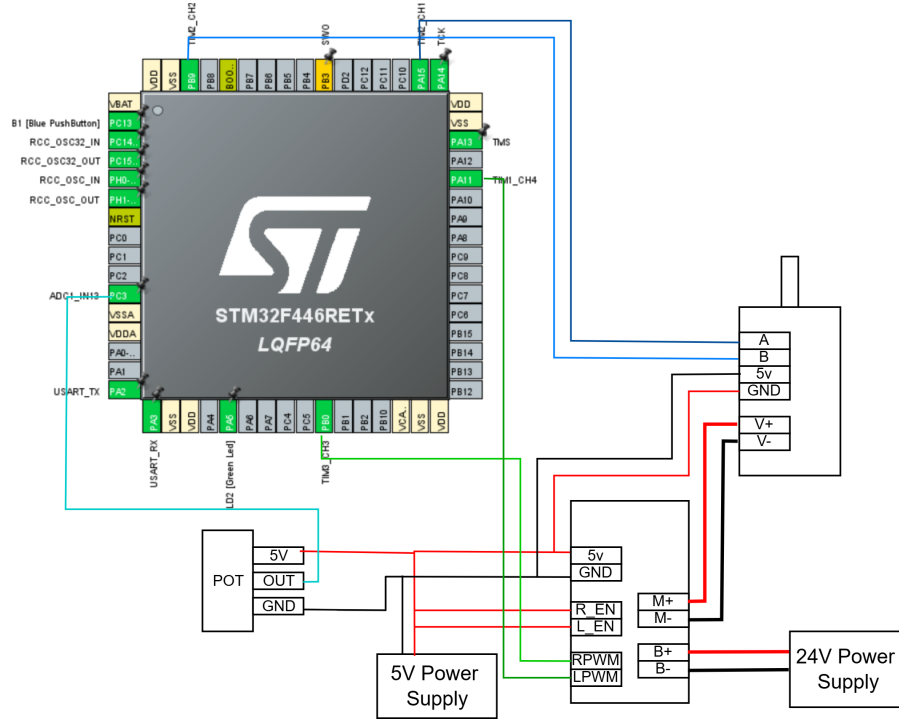


Figure 7: Diagram of Electrical Connections

Figure 7 displays an electrical diagram of the entire system. There are three inputs and two outputs to an STM32 microprocessor.

A potentiometer is used to generate the goal position and velocity signal. It is recorded using an analog in at port PC3. The encoder signal is recorded using the STM32 encoder feature through timer 2. The encoder channels A and B are connected to PA15 and PB9 respectively.

PWM is generated with timers 3 and 4, and are output on PB0 and PA11 respectively. PB0 controls forward rotation and PA11 controls backward rotation. A BTS7960 motor controller was used. A link to the motor driver datasheet can be found in the Appendix.

4 Controller Results

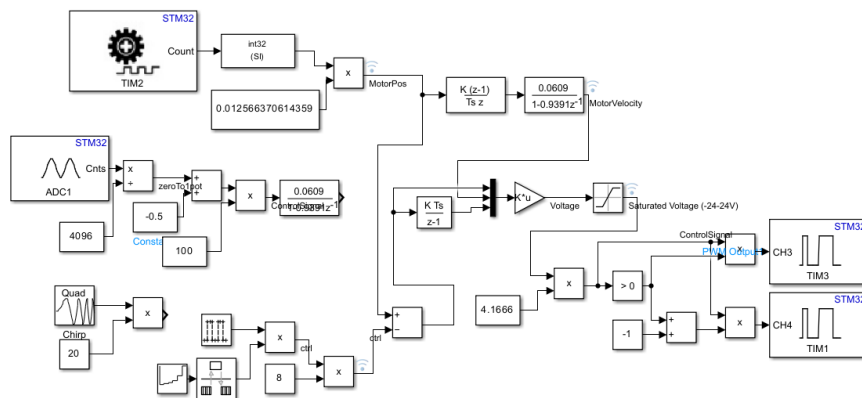


Figure 8: Simulink Position Controller

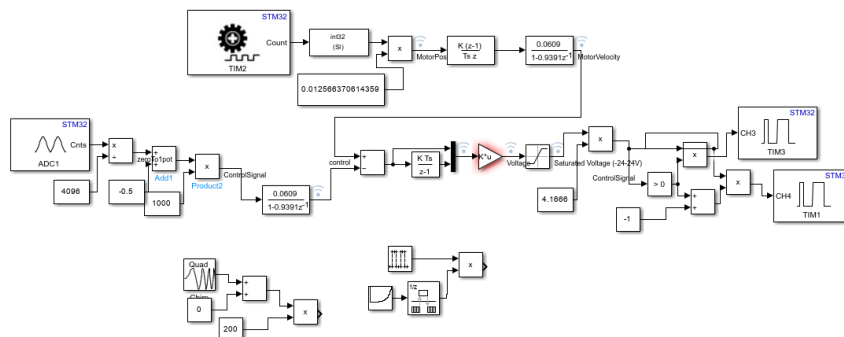


Figure 9: Simulink Velocity Controller

Figures 8 and 9 contain the Simulink models for position and velocity control. The encoder signal is read using a hardware timer in the STM32. The STM32 has a dedicated quadrature encoder interface which only requires connecting A and B to the STM32. This interface reads the encoder signal at a very high rate, which was convenient for this project as the Simulink model could be run at 1 kHz while still reading high speed encoder data. The closed loop rate for both controllers is 1 kHz.

ADC1 reads the user potentiometer input. This signal is filtered with a 10 Hz low pass. The derivative of motor position gives motor velocity and is filtered with a 10 Hz low pass.

Both the velocity and position controller largely use the same configuration in Simulink.

4.1 Position Control

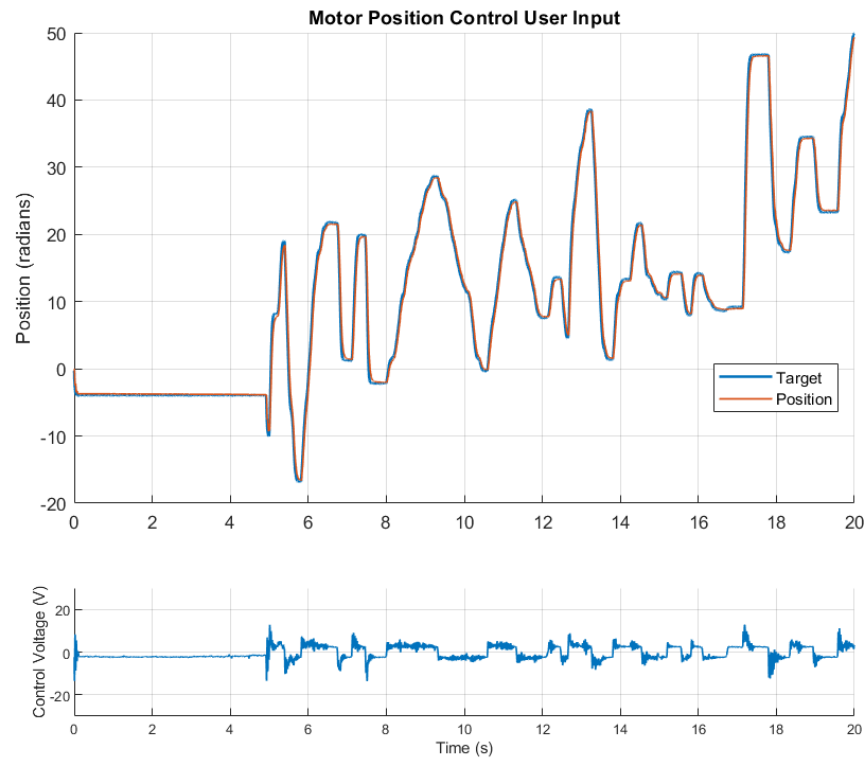


Figure 10: User Input Position Control

Figure 10 displays the position control from user input. The motor closely follows the input from the potentiometer and has minimal error. This shows that with reasonable commands, the controller has no issue following.

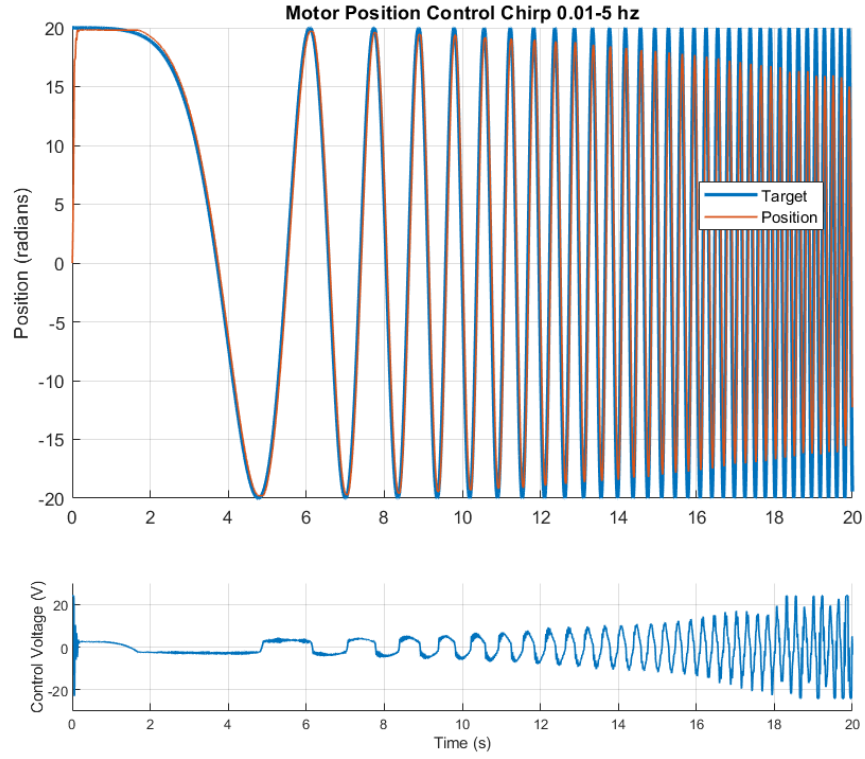


Figure 11: Position Control Chirp 0.01-5 Hz

Figure 11 shows the chirp response of the motor controller. The motor is able to follow, but does not reach the entire amplitude of 20 radians beyond approximately 1.5 Hz. The error increases as the frequency approaches 5 Hz.

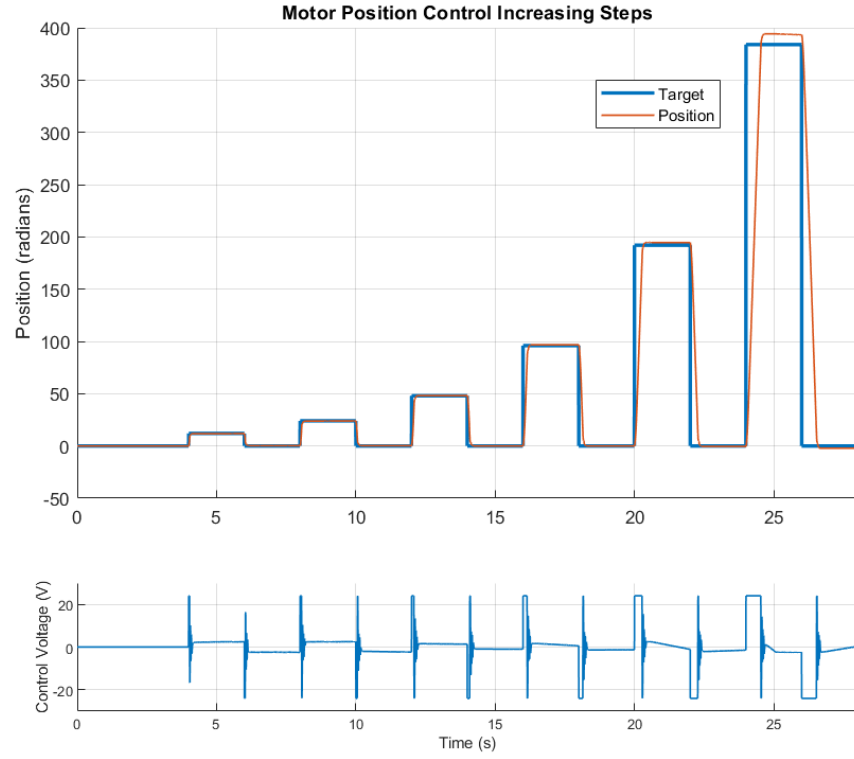


Figure 12: Position Control Increasing Steps

The limit of the motor movement is reached with increasing steps. As the step size increases, the motor takes more time to reach the desired position, limited by the motors maximum speed. Also, the error increases as the step size increases. This result is very similar to the simulated result in figure 3.

4.2 Velocity Control

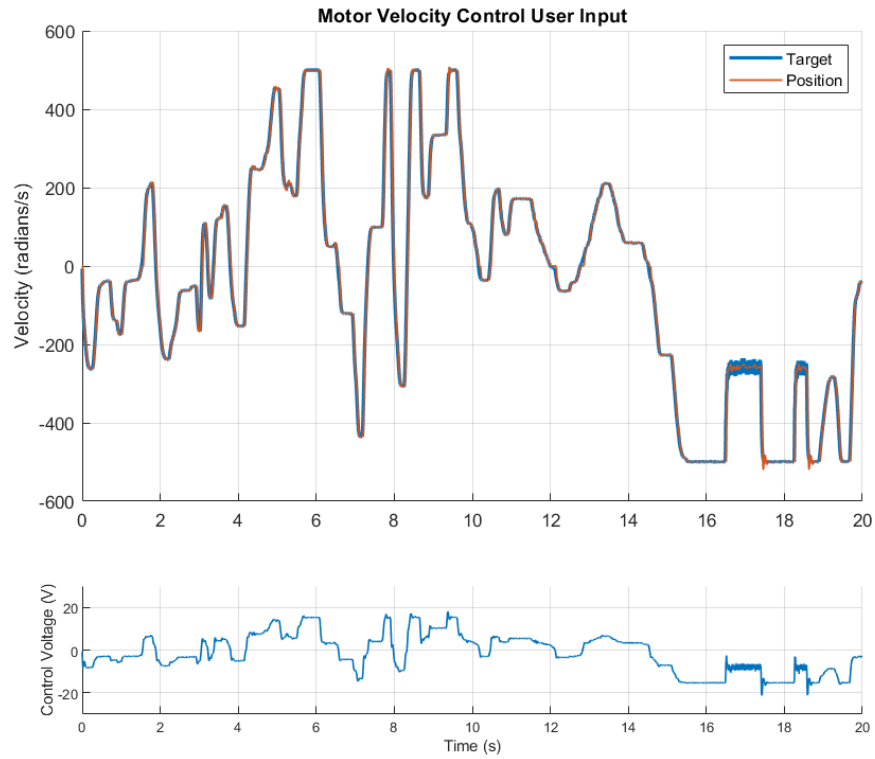


Figure 13: User Input Velocity Control

Similar to the position control, the velocity control with user input is able to follow very closely. This shows how responsive the motor is when given realistic commands.

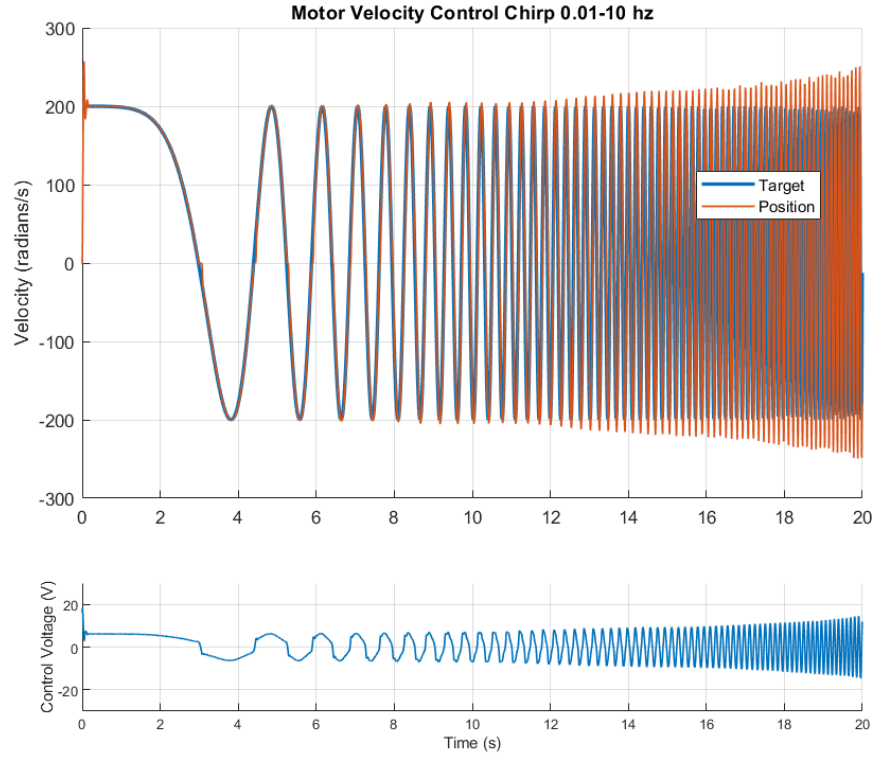


Figure 14: Velocity Control Chirp 0.01-10 Hz

Unlike the chirp simulation, the real chirp response has large overshoot at higher frequencies. A possible cause of this could be inaccurate modeling of the motor, with less velocity drop off at higher current.

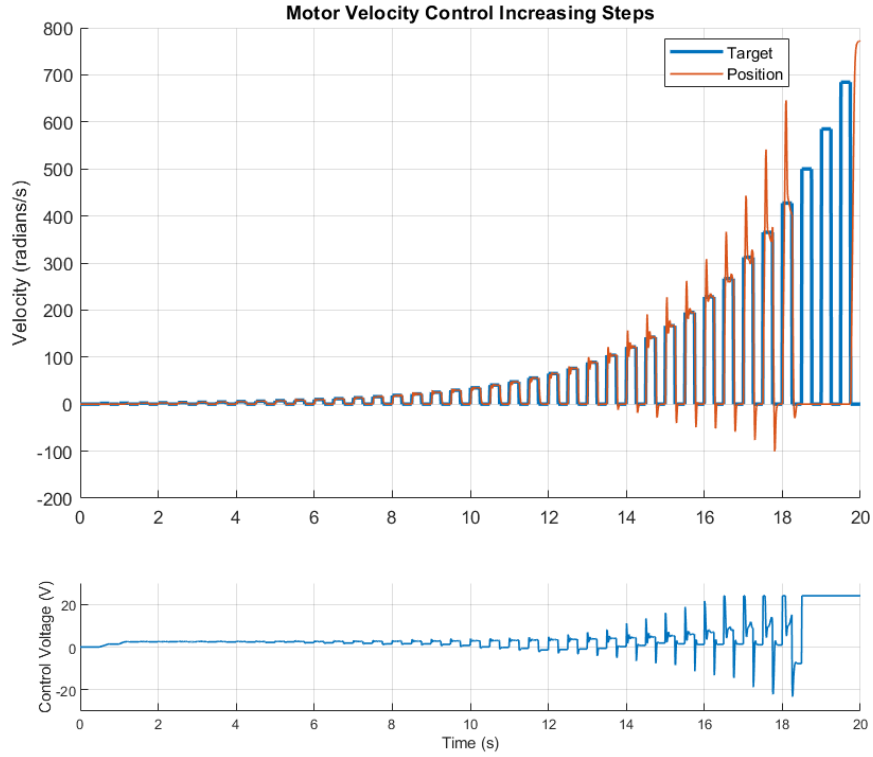


Figure 15: Velocity Control Increasing Steps

Velocity overshoot can also be seen in figure 15, further indicating that the motor model is inaccurate. Figure 6 shows a maximum motor velocity around 300 rad/s, with this experiment going far past that. Another issue can be seen in this simulation. If the motor direction switches too quickly, the H-Bridge stops working. This can be seen around 18 seconds into this experiment.

5 Appendix

BTS7960 Datasheet: <https://www.handsontec.com/dataspecs/module/BTS7960%20Motor%20Driver.pdf>

5.1 MATLAB Code

```
1  clc;
2  clear;
3
4  Ts = 0.001;
5
6  A = [0, 1;
7       0, -25.7909];
8
9  B = [0;
10      401.9964];
11
12 C = eye(2);
13 D = zeros(2,1);
14
15 plant1 = ss(A, B, C, D);
16
17 plant1_d = c2d(plant1, Ts, 'zoh');
18
19 [A1d, B1d, C1d, D1d] = ssdata(plant1_d)
20
21 % Original states:
22 % theta_dot = omega
23 % omega_dot = -25.7909*omega + 401.9964*u
24
25 % Define new augmented states:
26 % x1 = e = theta - ref
27 % x2 = omega
28 % x3 = integral(e) = integral(theta - ref)
29
30 Aa = [0, 1, 0;
31       0, -25.7909, 0;
32       1, 0, 0];
33
34 Ba = [0;
35      401.9964;
36      0];
37
38 Ca = eye(3);
39 Da = zeros(3,1);
40
41 plant_aug = ss(Aa, Ba, Ca, Da);
42
43 % Discretize augmented system
44 plant_aug_d = c2d(plant_aug, Ts, 'zoh');
45
46 [Ad, Bd, Cd, Dd] = ssdata(plant_aug_d);
47
48 % LQR weights
49 Q = diag([1e8, 1e3, 1e6]);
50 R = 1e6;
51
52 k = dlqry(Ad, Bd, Cd, Dd, Q, R)
```

Listing 1: MATLAB code used for generating position K matrix


```

1  clc;
2  clear;
3
4  Ts = 0.001;
5
6  Ts = 0.001;
7
8  A = [0, 1;
9       0, -25.7909];
10
11 B = [0;
12      401.9964];
13
14 C = eye(2);
15 D = zeros(2,1);
16
17 plant1 = ss(A, B, C, D);
18
19 plant1_d = c2d(plant1, Ts, 'zoh');
20
21 [A1d, B1d, C1d, D1d] = ssdata(plant1_d);
22
23 % Original velocity dynamics:
24 % omega_dot = -25.7909*omega + 401.9964*u
25
26 A = -25.7909;
27 B = 401.9964;
28
29 % Augmented states:
30 % x1 = e = omega - omega_ref
31 % x2 = integral(e) = integral(omega - omega_ref)
32
33 Aa = [A, 0;
34       1, 0];
35
36 Ba = [B;
37       0];
38
39 Ca = eye(2);
40 Da = zeros(2,1);
41
42 plant_aug = ss(Aa, Ba, Ca, Da);
43
44 % Discretize
45 plant_aug_d = c2d(plant_aug, Ts, 'zoh');
46
47 [Ad, Bd, Cd, Dd] = ssdata(plant_aug_d);
48
49 % LQR weights
50 Q = diag([1e3, 1e7]); % [velocity error, integral error]
51 R = 1e6;
52
53 K = dlqry(Ad, Bd, Cd, Dd, Q, R)

```

Listing 2: MATLAB code used for generating velocity K matrix